

Programa 4. Algoritmo de Planificación FCFS Continuación

Sistemas Operativos

Actividad 8 – Programa 4

Alumno: Quijas Contreras Victor Manuel

Código: 220429771

Carrera: ICOM **Sección:** D05

Profesor: Becerra Velázquez Violeta Del Rocío

Departamento: Ciencias Computacionales

Fecha: 29/03/2026

Índice

Índice	1
Tabla de ilustraciones	1
Objetivo	1
Desarrollo	1
Función de nuevos procesos	1
Tabla BCP	3
Conclusión	3

Tabla de ilustraciones

Ilustración 1. Case que controla función de teclas.	2
Ilustración 2. Funciones adicionales	2
Ilustración 3. Tabla BCP	3

Objetivo

El objetivo de esta práctica es el de construir el de implementar la función de crear nuevos procesos durante la ejecución del programa, es decir que la cantidad de procesos inicial se mantendrá, pero podrá aumentar conforme el usuario lo desee. Y la implementación de la tabla de bloque de control de procesos (BCP) la cual muestra la información de cada uno de los procesos que se encuentran en el sistema: nuevos, en memoria, ejecución, bloqueados y terminados.

Desarrollo

Para el desarrollo de las funciones nuevas se realizaron cambios en la clase Tabla y la implementación de una clase extra la cual es un JFrame que muestra la tabla BCP.

Función de nuevos procesos

La función para ingresar nuevos procesos se divide entre distintas verificaciones que se necesitan realizar para decidir donde el programa debe poner al proceso nuevo. Existen tres posibles casos:

Programa 4. Algoritmo de Planificación FCFS Continuación

1. **Memoria llena:** La memoria actualmente se encuentra llena y solo coloca el proceso en la lista de procesos.
2. **Memoria con espacios:** Si la memoria cuenta con al menos un espacio, el proceso nuevo se agrega además de a la lista de procesos, a listos y se muestran los datos pertinentes.
3. **Memoria vacía:** Si la memoria está vacía debido a que todos los procesos se fueron a bloqueo, entonces el proceso nuevo entra directo a ejecución.

Dentro del programa esto se ve de la siguiente manera. Si la tecla 'n' fue presionada, entonces el contador de procesos incrementa. La simulación se encarga de verificar que el contador de procesos sea igual al tamaño de la lista, si no es así entonces, crea procesos hasta que la condición se satisface.

```
    }  
    break;  
    case KeyEvent.VK_N:  
        if (reloj.isRunning()) {  
            numListaProcesos++;  
        }  
        break;
```

Ilustración 1. Case que controla función de teclas.

```
private void AgregarProcesoAListos() {  
    if (i < listaProcesos.size()) {  
        procesosListos.agregarProceso(listaProcesos.get(i)); // <-- Añade un nuevo proceso en la lista  
        listaProcesos.get(i).setTiempoLlegada(contadorGlobal - 1);  
        procesosLlegados++; // <-- Aumenta la cantidad de procesos que pasaron por el núcleo del sistema  
        i++;  
    }  
}  
  
private void AgregarProcesoALista() {  
    listaProcesos.add(CrearProceso());  
}  
  
private Proceso CrearProceso() {  
    Proceso p = new Proceso(  
        "Proceso-" + procesoId,  
        procesoId,  
        (int) (Math.random() * 6), // Operación (0 a 5)  
        (int) (Math.random() * 50), // X  
        (int) (Math.random() * 50) + 1, // Y (evitamos división por 0)  
        (int) (Math.random() * 15) + 6  
    );  
    procesoId++;  
    return p;  
}  
  
private void ChecarNuevos() {  
    if (numListaProcesos > listaProcesos.size()) {  
        int j = listaProcesos.size();  
        while (j < numListaProcesos) {  
            AgregarProcesoALista();  
            if (procesosListos.tamano() + procesosBloqueados.size() < 5)  
                AgregarProcesoAListos();  
            j++;  
        }  
    }  
}
```

Ilustración 2. Funciones adicionales

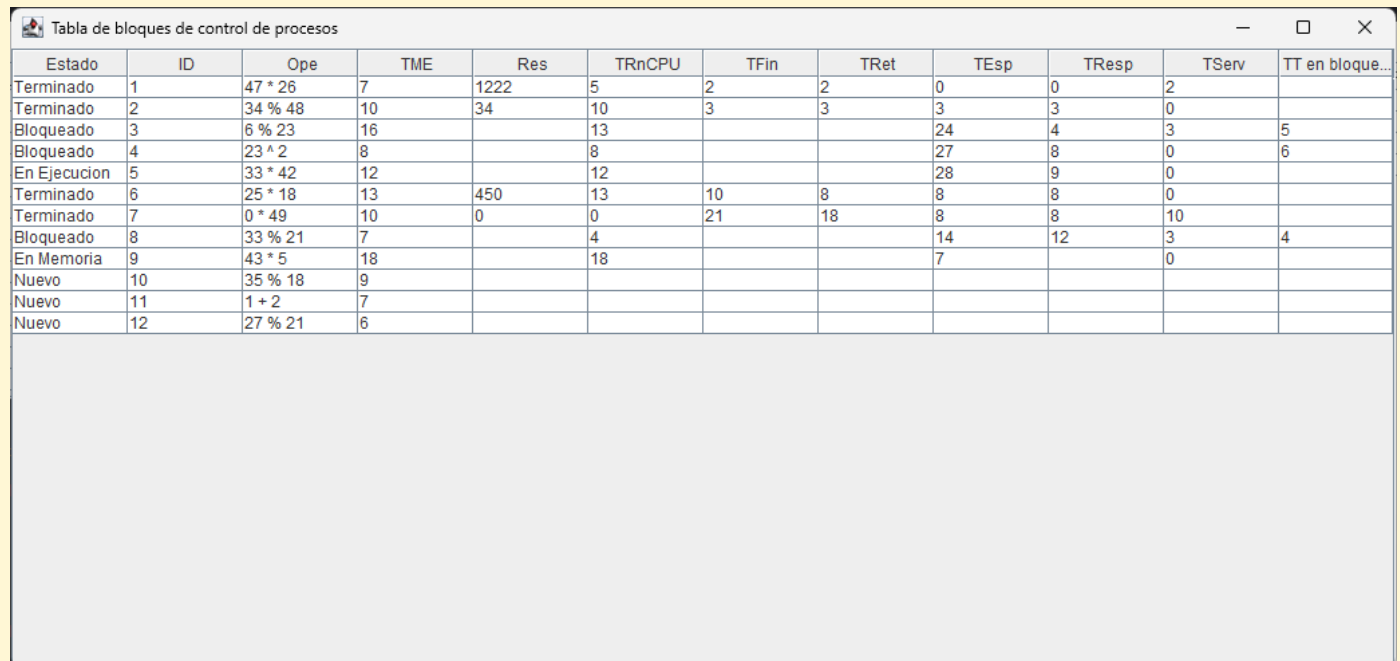
Tabla BCP

Para desarrollar la tabla BCP, la tabla debe de recibir la lista de procesos, la lista de procesos listos y el contador global.

La estructura de la tabla se basa en dos funciones principales las cuales son: `getEstado()` y `MostrarBCP()`.

La función `getEstado` se encarga de decidir si un proceso se encuentra en ejecución, en memoria, bloqueado o terminado, basándose en los atributos de cada proceso.

Y la función `MostrarBCP` se encarga de desplegar la información de cada proceso dependiendo el estado en el que se haya mandado.



Estado	ID	Ope	TME	Res	TRnCPU	TFin	TRet	TEsp	TResp	TServ	TT en bloque...
Terminado	1	47 * 26	7	1222	5	2	2	0	0	2	
Terminado	2	34 % 48	10	34	10	3	3	3	3	0	
Bloqueado	3	6 % 23	16		13			24	4	3	5
Bloqueado	4	23 ^ 2	8		8			27	8	0	6
En Ejecucion	5	33 * 42	12		12			28	9	0	
Terminado	6	25 * 18	13	450	13	10	8	8	8	0	
Terminado	7	0 * 49	10	0	0	21	18	8	8	10	
Bloqueado	8	33 % 21	7		4			14	12	3	4
En Memoria	9	43 * 5	18		18			7		0	
Nuevo	10	35 % 18	9								
Nuevo	11	1 + 2	7								
Nuevo	12	27 % 21	6								

Ilustración 3. Tabla BCP

Conclusión

Con estas dos nuevas implementaciones el simulador es capaz de emular el comportamiento del algoritmo FCFS de una forma más completa. La herramienta de la tabla BCP es de una utilidad imprescindible pues registra todos los tiempos que tiene cada proceso y rastrea cual es su estado actual.